

noisefloor.ca — Project Outline

Case-based training for fixed-wireless ISP support technicians. Learn to read the instruments, not memorize the answers.

This document is the founding brief for the `noisefloor` repository. It is written to be read cold by a developer (human or Claude Code) who has never seen the conversation that produced it. Everything a first `openspec` proposal needs should be derivable from here. Where a decision is still open it is marked **[DECIDE]** and listed again in §14.

1. What this is

A public website where support trainees work through **cases**: real-shaped support tickets told in stages. Each stage reveals one more piece of evidence (a dashboard chart, a radio link panel, a ping log, a customer's words), and the trainee must commit to a hypothesis or a next action before the next stage unlocks. The skill being trained is not "know the answer" but **revise a theory as evidence arrives, and say what you touched**.

The domain is fixed-wireless ISP support (Ubiquiti airMAX / UISP / a typical WISP billing-CRM), because:

- It is narrow enough for one person with domain knowledge to own.
- Every WISP in Canada and the US has Tier 1 trainees and almost no training material beyond vendor docs.
- The instruments (link capacity charts, signal-over-time, modulation bars, pinglogs) are visual and learnable, and nobody teaches how to read them.

The name comes from the first number a good tech learns to read. The product promise is the same thing: separate the signal from what's just there.

2. Who it is for

Primary: Tier 1 support trainees at fixed-wireless ISPs, first 0–12 months. They can log into the CRM and the radio but do not yet know what the graphs mean or what changed.

Secondary: Tier 1s who want to become Tier 2. Cases have optional deeper branches (RF theory, config rollback, channel planning) for them.

Tertiary (later): ISP training leads who want to assign cases to a cohort and see who is guessing vs reasoning. This is the eventual paid tier and is **out of scope for v1** — but the data model should not make it impossible.

Not for: generic call-centre / helpdesk training. No "how to handle an angry customer" as a standalone module. Customer communication appears only inside a technical case, as a stage.

3. Product principles

1. **Evidence is a dashboard, not a paragraph.** Trainees must read the same instruments they'll read at work. A case never says "signal is -70 dBm"; it shows the panel that says it.
2. **Commit before reveal.** No stage unlocks without a stated hypothesis or action. Skipping ahead is not possible.
3. **Reward revision, not just the final answer.** A trainee who is wrong at stage 1 and correct by stage 3 has demonstrated the skill. Scoring reflects the path.
4. **"What changed, and when?"** is a first-class prompt that appears in every case. Most real tickets fall to it.
5. **Some cases have no fix.** "Monitor, note, schedule a survey" is a valid and sometimes correct final action. Trainees who always reach for a config change are dangerous.
6. **The customer call is a deliverable.** At least one stage per case asks what you'd say to the person. Judged on honesty and expectation-setting.
7. **Gotchas are indexed, not lectured.** Vendor-specific traps (units, test-vs-apply, memory in router mode) are tagged on cases and aggregate into a reference that grows from stories.
8. **Nothing real leaks.** No real customer names, IPs, MACs, coordinates, tower names, or employer branding. See §12.

4. Core concept: dashboards are components, cases are data

This is the architectural spine.

A case does **not** contain screenshots. It contains **structured data** — signal values, time series, a station list, a shaper value — and the site renders synthetic dashboards from that data using a library of React components that look like the real tools.

Consequences:

- Authoring a new case is editing JSON (or TS), not producing artwork.
- A case variant ("same story, foliage loss 3 dB worse") is a one-field change.
- Data can be generated: a foliage curve, a diurnal usage pattern, a shaper collapse are all functions of a few parameters.
- Components are testable in isolation with Vitest and visually with Playwright screenshots.
- The real vendor screens are used only as **private, uncommitted visual reference** during component development.

The three tool families to recreate (name each family neutrally in code; do not use vendor names in user-facing copy unless licensing is confirmed — **[DECIDE]** see §14):

Family	Internal name	What it shows	Real-world analogue
Billing/CRM	crm	customer header, service tabs, Link Capacity stacked chart + signal trace, Summary with pinglog heatmap, realtime ping modal, bandwidth history	typical WISP CRM (Sonar/Splynx/Powercode-style)
Radio link view	radio	local/remote device cards, throughput gauges, RF environment bar, signal + chain bars, modulation rate bar (1X–8X), capacity/throughput/latency chart, device details (mode, uptime, CINR, noise floor, cable SNR, memory/CPU)	airOS 8 link view
Network controller	nms	device overview (throughput charts, utilization, link potential/capacity), Manage pane (backups, advanced), AP station list with airtime/capacity/connection time, RF scan	UISP Network

Each family is a set of components in `packages/dashboards` (see §7).

5. Case model

5.1 Case

```
type Case = {
  id: string;                // "c001-wet-leaves"
  slug: string;
  title: string;             // "It's slow when it rains"
  version: number;
  difficulty: 1 | 2 | 3;
  estimatedMinutes: number;
  tags: string[];           // ["foliage", "shaping", "units", "router-mode"]
  gotchas: GotchaId[];      // links into the gotcha index
  world: World;             // all synthetic data, see 5.2
  opening: Opening;         // ticket text + first evidence
  stages: Stage[];          // ordered
  debrief: Debrief;         // the full story, told after completion
  optionalBranches?: Branch[]; // deeper dives, unlocked after debrief
};
```

5.2 World

All the numbers a case needs, in one object, so every dashboard renders from the same truth.

```
type World = {
  customer: { displayName: string; accountRef: string; plan: { down: number; up: number } };
  site: { apName: string; sectorName: string; channelMHz: number; widthMHz: number };
  cpe: Device;                // model, mode, firmware, uptime, memoryPct, cpuPct, cableSnrDb, cable
  ap: Device;
  link: {
    distanceM: number;
    signalLocalDbm: number; signalRemoteDbm: number;
    chainsLocal: [number, number]; chainsRemote: [number, number];
    noiseFloorLocalDbm: number; noiseFloorRemoteDbm: number;
    cinrLocalDb: number; cinrRemoteDb: number;
    rateLocal: 1|2|3|4|5|6|7|8; rateRemote: 1|2|3|4|5|6|7|8;
    capacityDownMbps: number; capacityUpMbps: number;
    latencyMs: number;
    linkPotentialPct: number;
    airtimeTxPct: number; airtimeRxPct: number;
  };
  series: {
    // each is an array of { t: ISO8601, v: number } or a generator spec (see 5.3)
  }
};
```

```

    capacityDown24h: SeriesRef;
    usedDown24h: SeriesRef;
    signalTrace24h: SeriesRef;
    capacityDown1y: SeriesRef;
    signalTrace1y: SeriesRef;
    throughputRx1h: SeriesRef;
    pinglog: PinglogRef;      // day × minute grid, values: ok | slow | loss
};
stationList: StationRow[]; // the AP's other customers (synthetic)
events: WorldEvent[];      // timestamped things that happened: reboot, config change, etc.
};

```

5.3 Series generators

Time series are expensive to hand-author and cheap to generate. `packages/shared/src/gen/` holds pure functions:

- `diurnalUsage({ peakMbps, peakHour, offHours, noise })` — residential download pattern.
- `noisyCeiling({ base, jitter, spikes })` — a capacity ceiling that bounces.
- `foliageYear({ leafOnDbm, leafOffDbm, leafOnDate, leafOffDate, growthDbPerYear })` — seasonal signal trace.
- `shaperCollapse({ at, toKbps })` — throughput drops to a trickle at a timestamp.
- `pinglogMonth({ baseLossPct, outages: [{ start, minutes }], eveningSpeckle })`.

A `SeriesRef` is either inline points or `{ gen: "foliageYear", params: {...} }`. Rendering resolves generators with a seeded RNG so the same case always draws the same chart.

5.4 Stage

```

type Stage = {
  id: string;
  title: string;
  reveal: Evidence[];      // what becomes visible at this stage
  prompt: Prompt;          // what the trainee must commit to
  rubric: Rubric;          // how the commitment is scored
  feedback: Feedback;      // shown after commitment, before next stage
};

type Evidence =
  | { kind: "dashboard"; family: "crm"|"radio"|"nms"; view: string; worldSlice?: string; annotate?:
  | { kind: "customerSays"; text: string }

```

```

| { kind: "ticketNote"; author: string; text: string; at: string }
| { kind: "colleagueSays"; role: "T1"|"T2"|"field"; text: string };

type Prompt =
| { kind: "hypothesis"; options: Option[]; allowFreeText: true }
| { kind: "nextCheck"; options: Option[] }           // "what would you look at next?"
| { kind: "whatChanged"; freeText: true }           // always includes a time field
| { kind: "customerMessage"; freeText: true; minWords: number }
| { kind: "ticketNote"; freeText: true }
| { kind: "action"; options: Option[] };           // includes "do nothing today" where valid

```

5.5 Rubric and scoring

- Option prompts: each option has `score: 0..3` and `feedback`. Wrong-but-reasonable options score 1–2 and say why they're reasonable.
- Free-text prompts (hypothesis, whatChanged, customerMessage, ticketNote): v1 scores by **keyword/phrase rubric** (`mustMention`, `mustNotMention`, `bonus`). An LLM grader is a **v2 consideration**, not v1 — keep the rubric shape LLM-friendly (criteria as sentences) so it can be swapped in.
- Path scoring: total = Σ stage scores, plus a **revision bonus** when a later hypothesis corrects an earlier wrong one. A trainee who was wrong at stage 1 and right by stage 3 can outscore one who was lucky at stage 1 and never restated.
- Customer-message rubric criteria are things like: says nothing is broken if true; sets a timeline; does not promise a fix that isn't scheduled; owns a mistake if the case contains one.

5.6 Debrief

Shown after the final stage. The full story in prose, with each dashboard re-shown **annotated** (callouts on the exact feature the evidence hinged on). Lists the gotchas hit. Links to optional branches.

5.7 Gotcha index

```

type Gotcha = { id: string; title: string; oneLiner: string; explanation: string; cases: string[] }.

```

Seed gotchas from case 001 (see §9). The index page lists all gotchas with the cases that contain them. This is the "reference manual that grew from stories."

6. Repository and stack

[DECIDE] New repository (`noisefloor`) vs. new apps inside the existing ticketing-app monorepo. Recommendation: **new repo**, same stack, same conventions. The ticketing app is an internal-to-employer tool; noisefloor is a public product with a different audience, deploy cadence, and (eventually) billing. Shared conventions can be copied; shared code should not be.

Stack (mirror of the ticketing app, chosen for zero ramp-up):

- **Monorepo:** pnpm workspaces, TypeScript throughout.
- **Frontend** `apps/web` : React 19, React Router 8, Vite 8, Tailwind CSS v4, Lucide icons. Playwright for e2e and for **component visual snapshots** (used to keep dashboards looking right).
- **Backend** `apps/api` : Fastify 5, Drizzle ORM + Postgres (`pg`), Better Auth, Zod.
- **Packages:**
 - `packages/shared` — Zod schemas for Case/World/Stage/etc., series generators, seeded RNG, scoring functions. Pure, no React, no DB. Everything here is unit-tested.
 - `packages/dashboards` — the React component library for `crm` , `radio` , `nms` . Depends on `shared` for types. Has its own Storybook-less "gallery" route in `apps/web` (`/dev/gallery`) for visual development against reference screenshots.
 - `packages/cases` — the case content, one folder per case, exported as typed objects validated against `shared` schemas at build time.
- **Tooling:** Vitest, ESLint + typescript-eslint, tsx. **OpenSpec** for spec-driven changes — every feature below should become an `openspec` proposal before code.
- **Infra:** Railway (API + Postgres), Vercel (web). Domain: `noisefloor.ca` (registered, WHC). Web at apex; API at `api.noisefloor.ca` .

Why cases are a package and not DB rows in v1: cases are authored by the developer, version-controlled, reviewed in PRs, and validated at build. The DB holds **users and attempts**, not content. If a case-authoring UI is ever built (§13), content can move; the Zod schema is the contract either way.

7. `packages/dashboards` — component inventory

Each component takes a `world` (or a slice) and renders. All are read-only in v1 except where a stage explicitly needs an interactive element (e.g. a shaper input in a "what would you set?" prompt).

Visual fidelity target: **recognizably the tool** to someone who uses it daily, without being a pixel copy. Use the tool's layout and information hierarchy; use our own palette, type, and icons. See §12 on trademarks.

crm/

- `CustomerHeader` — name, account ref, status chips.
- `ServiceBar` — service label + tab strip (Edit / Speedometer / Bandwidth History / Link Capacity / IPs / Summary).
- `LinkCapacityChart` — stacked bars (used + remaining), optional package-limit line, period selector (24h / 1y), with signal trace panel beneath. **The single most important component in v1.**
- `PinglogHeatmap` — day rows × minute columns, ok/slow/loss colouring, month selector.
- `RealtimePingModal` — bar chart of RTT, loss %, avg, source/target.
- `SummaryPanel` — AP name/IP, radio login, CPE-issue banner with task/timestamp.

radio/

- `LinkHeader` — local/remote device cards (model, coords redacted/synthetic, MAC synthetic, TX power), throughput/capacity gauges, distance, link potential, airtime.
- `RfEnvironmentBar` — spectrum occupancy strip with current channel marker.
- `SignalPanel` — signal dBm with per-chain bars and Δ , noise floor.
- `RateBar` — 1X–8X modulation bar with expected vs actual.
- `LinkChart` — capacity / throughput / latency over time.
- `DeviceDetails` — model, mode (router/bridge), firmware, uptime, memory %, CPU %, wireless (CINR, distance, noise floor), ethernet (LAN speed, cable SNR with red-below-threshold, cable length), GPS (AP only).

nms/

- `DeviceOverview` — throughput charts per interface, utilization %, link potential + capacity, "Link Needs Improvement" style status.
- `DeviceManagePane` — device name, subscriber assignment, **Backups** (list with timestamps, restore action), Advanced, speed test, maintenance mode.
- `ApStationList` — table: MAC, model, name, signal, remote signal, downlink/uplink capacity, airtime TX/RX, connection time, last IP, throughput RX/TX. Row highlight for "this customer."
- `ApLinkView` — the AP-side version of `radio/LinkHeader` with average capacity chart.

Shared chart primitives

Keep one internal chart layer (SVG, no heavy lib) in `dashboards/primitives/`: `StackedBars`, `LineTrace`, `Heatmap`, `Gauge`, `SegmentBar`. Tailwind for layout, inline SVG for plots. Avoid pulling in a charting library unless a primitive proves painful.

Annotation mode

Every dashboard component accepts `annotations?: Annotation[]` — callouts anchored to a data feature (a timestamp, a bar, a row). Used in debriefs and in feedback. `{ target: { series: "signalTrace1y", t: "2026-05-24" }, label: "Leaf-out. -8 dB in four days." }`.

8. apps/web — routes

<code>/</code>	landing: what this is, one demo stage playable without login
<code>/cases</code>	case list (difficulty, tags, time, completion state if logged in)
<code>/cases/:slug</code>	case player (stages, commit-before-reveal)
<code>/cases/:slug/debrief</code>	annotated debrief (locked until completion)
<code>/gotchas</code>	gotcha index
<code>/gotchas/:id</code>	gotcha detail + cases
<code>/me</code>	progress, attempts, scores
<code>/auth/*</code>	Better Auth routes
<code>/dev/gallery</code>	component gallery (dev only, not deployed to prod)

Case player behaviour:

- Left: evidence panel (accumulates; earlier evidence stays visible, collapsible).
- Right: current prompt. Commit is required; there is no skip.
- After commit: feedback for that stage, then "Continue."
- Progress persists per attempt; a trainee can leave and return.
- Anonymous users can play case 001 stage 1 only (the landing demo); login required beyond.

9. Seed content: Case 001 — "It's slow when it rains"

Fully synthetic. Any resemblance to real customers is by design of the genre only.

Tags: foliage, seasonal-signal, chain-imbalance, shaping, units, router-mode-memory, what-changed, own-your-change **Difficulty:** 2 · ~25 min

World (headline values):

- Customer "M. Ferrier", account `SYN-00417`, plan 35/10 → later 50/10.
- Site: AP `SEC-A`, sector `RIDGE-5.8-A`, 5710 MHz, 20 MHz.
- CPE: PowerBeam-class 25 dBi dish, **Router mode**, memory 78 %, CPU 20 %, uptime 2d 18h, cable SNR +27 dB (red), 28 m.
- AP: sector, Bridge mode, uptime 30 d, cable SNR +30 dB, GPS 9 sats.
- Link: 402 m; signal -69 local / -66 remote; chains $\Delta 5$ / $\Delta 6$; noise floor -104 / -91; CINR +23 / +21; rate 6X both; capacity 66 down / 93 up; latency 1 ms; link potential 52 %; airtime 11 %.
- Another station on the AP: "Lakeside Inn", NanoStation-class, -51 dBm, 140/148 capacity, airtime 1.8 %.
- Series: 24 h capacity noisy 55–80 with usage peaking 35 Mbps at 22:18 then dropping to ~0 at 02:18; 1 y signal via `foliageYear({ leafOnDbm: -72, leafOffDbm: -65, leafOnDate: "05-24", leafOffDate: "10-20", growthDbPerYear: 3 })`; pinglog with three short outages and one degraded evening in August; 1 h throughput via `shaperCollapse({ at: "11:56", toKbps: 50 })`.
- Events: `11:21 UISP reconnect`, `11:52 config change on CPE by T1 (shaper)`, `11:56 throughput collapse`, `12:34 T2 restore`, `12:35 CPE reboot`.

Stages (in order):

1. **"What does this graph tell you about capacity?"** — reveal `crm/LinkCapacityChart` (24 h). Prompt: hypothesis. Rubric: green+blue = capacity; usage never near ceiling; ceiling noisy; flat top ~60 in the night window is a plausible cap.
2. **"Why is the ceiling where it is?"** — reveal `radio/LinkHeader`, `SignalPanel`, `RateBar`. Prompt: nextCheck (options: realign / widen channel / check signal history / replace radio / nothing yet). Rubric: 20 MHz + 6X + TDD explains the number; chain imbalance and -69 at 400 m are the anomaly; correct next check is history, not a truck.
3. **"Should this be a dispatch?"** — reveal `radio/DeviceDetails` (cable SNR red, memory 78 %, router mode, uptime mismatch). Prompt: action. Rubric: not on its own; note items; bundle if a truck goes anyway.
4. **"This feels telling."** — reveal `crm/LinkCapacityChart` (1 y). Prompt: hypothesis (free text). Rubric: seasonal → foliage; late-May cliff; year-over-year 3 dB worse → growth; realignment won't fix it; survey for mast height/trimming before spring.
5. **"What do you tell the customer?"** — Prompt: customerMessage. Rubric: nothing broken; seasonal; wet leaves absorb signal (not "weigh down"); what happens next; still above current usage; does not overpromise.

6. **"You changed the plan."** — reveal ticketNote (T1 set shaper on CPE), then colleagueSays ×3 (techs can't hold a session). Prompt: hypothesis (options include "blown horn / hardware", "shaper misconfigured", "memory", "AP problem"). Rubric: hardware doesn't wait for a config change; ask what changed and when.
7. **"Is she online?"** — reveal `crm/RealtimePingModal` (65 ms, 1 % loss) and same for Lakeside Inn (69 ms, 3 %). Prompt: whatChanged. Rubric: ping proves almost nothing about throughput; baseline is the neighbour; the only change is the shaper at 11:52.
8. **"I haven't done anything but I see traffic."** — reveal `nms/DeviceOverview` (1 h, collapse to ~50 kbps, gaps where NMS lost contact, symmetric MB totals). Prompt: hypothesis. Rubric: trickle not service; symmetric traffic = management chatter; gaps = starved management; shaper in **kbit/s**.
9. **"Get it back."** — reveal `nms/DeviceManagePane` (Backups section collapsed). Prompt: action (options: keep trying the web UI / restore NMS backup / SSH and disable shaper / power-cycle again / escalate to T2 with the exact change). Rubric: backup restore or SSH; escalating with the exact change is full marks too; web UI over 50 kbps is the wrong tool.
10. **"It's back."** — reveal `nms/ApStationList` post-fix (30 Mbps RX, 8 ms, connection time 00:38). Prompt: ticketNote. Rubric: cause in kbit/s vs Mbps; window of outage; corrected by whom; recommendation: shape centrally; foliage survey carried forward.

Debrief: the full narrative with annotated re-shows of stages 1, 4, 8, 10.

Optional branch A — "Would 40 MHz help?" Rubric: +3 dB noise floor cost; may drop to 5X; RF scan shows neighbours; AP-wide decision; dish job first. **Optional branch B — "Why is downlink 64.7 and uplink 92.6?"** Rubric: channel width × modulation × TDD share; per-direction error/retry estimate; vendor math, not derivable exactly.

Gotchas seeded from this case:

- `shaper-units-kbps` — airOS traffic shaping fields are kbit/s.
- `test-vs-apply` — Test reverts in ~3 min; Apply persists.
- `router-mode-memory` — CPE in router mode runs hot on RAM; config pushes can tip it.
- `chain-imbalance` — $\Delta > 3$ dB between chains means off-axis or partial obstruction.
- `ping-good-throughput-zero` — ICMP passes through a starved shaper; ping is not a throughput test.
- `seasonal-signal-is-trees` — signal that follows the calendar is foliage; that follows the weather is wet foliage.
- `what-changed-when` — before hardware theories, ask what changed and when.

- `own-your-change` — the fastest fix for a multi-tech chase is the person who touched it saying SO.
- `channel-width-is-ap-wide` — width is a sector decision, not a customer knob.
- `cable-snr-threshold` — cable SNR shown red under vendor threshold; marginal, not broken.

10. Data model (`apps/api` , Drizzle)

Users and attempts only. Content lives in `packages/cases` .

```
users          (Better Auth managed)
attempts       id, user_id, case_id, case_version, started_at, completed_at, total_score, path_json
stage_commits  id, attempt_id, stage_id, committed_at, prompt_kind, answer_json, score, feedback_json
gotcha_progress user_id, gotcha_id, first_seen_case_id, seen_count
```

`path_json` stores the ordered list of hypotheses so revision bonus can be computed and the "you were here → here → here" strip shown in `/me` .

11. API surface (`apps/api` , Fastify + Zod)

```
GET  /cases          list (public metadata only)
GET  /cases/:slug     case shell: opening + stage ids; NO rubrics, NO later-stage evidence
GET  /cases/:slug/stage/:id evidence for a stage – only if attempt has committed to prior stage
POST /attempts       start
POST /attempts/:id/commit { stageId, answer } → { score, feedback, nextStageId | debriefUntil
GET  /attempts/:id     progress
GET  /me/progress
GET  /gotchas, /gotchas/:id
```

Rubrics and future-stage evidence never leave the server. The client cannot peek ahead by reading the case bundle. (This is why `packages/cases` is imported by the API, not shipped to the browser — the web app receives stage payloads, not the case object.)

12. Privacy, legal, and taste rules

- **No real data.** Customer names, account numbers, IPs, MACs, coordinates, tower names, SSIDs, employer names: all synthetic. Case content is reviewed for this in PR. A `pnpm check:synthetic` script greps `packages/cases` for RFC-1918-looking-but-real IPs, MAC patterns, and a denylist of real names/tokens.
- **Reference screenshots** from real systems are kept **outside the repo** (`~/noisefloor-ref/`, gitignored path documented in README). Never committed, never deployed.
- **Vendor names and trademarks. [DECIDE]** whether to name vendors in copy ("Ubiquiti airOS", "UISP"). Recommendation for v1: describe generically in UI ("radio link view", "network controller", "CRM") and name vendors only in gotcha explanations where precision matters, with a plain "not affiliated" line in the footer. Revisit if a vendor relationship ever exists.
- **Employer.** The site is Robin's independent project. No employer branding, no employer data, no statements about a specific employer's network or practices. Cases are genre-typical, not employer-typical.
- **Tone.** Practical, dry, respectful of the trainee. Feedback explains why a wrong answer was reasonable before saying why it's wrong. No gamification noise (no streaks, no confetti). A score and a path are enough.

13. Phases

Phase 0 — Skeleton (get to a running monorepo)

- `pnpm workspace;` `apps/web`, `apps/api`, `packages/shared`, `packages/dashboards`, `packages/cases`.
- Zod schemas for \$5 in `shared`; seeded RNG; the five series generators with tests.
- Better Auth wired; empty Postgres via Drizzle; Railway + Vercel deploy of a "hello" page at `noisefloor.ca`.
- OpenSpec initialized; this document referenced from README.

Phase 1 — The one component that matters

- `crm/LinkCapacityChart` (24 h and 1 y) rendering from `World`, with annotation support.
- `/dev/gallery` route showing it against the generator outputs for case 001.
- Playwright visual snapshot for it.

Phase 2 — Case engine

- Case player route; commit-before-reveal; server-side stage gating; scoring incl. revision bonus; attempts persisted.

- Case 001 authored with stages 1, 4, 5 only (the foliage arc), using only `LinkCapacityChart` + text evidence. **Playable end-to-end.** This is the first thing worth showing anyone.

Phase 3 — Radio and NMS components

- `radio/*` and `nms/*` per §7, gallery entries, snapshots.
- Case 001 completed to all 10 stages + debrief + branches.

Phase 4 — Public v1

- Landing with anonymous stage-1 demo; `/cases`; `/gotchas`; `/me`.
- Gotcha index seeded from §9.
- Synthetic-data check script in CI.
- Footer: about, not-affiliated, contact.

Phase 5 — Content velocity

- Cases 002–005. Candidates: *interference from a new neighbour AP* (RF scan changes, capacity drops, signal unchanged); *"my speed test is slow" with a 100 Mbps LAN negotiation* (cable, not RF); *router-mode reboot loop* (memory); *the no-fix case* (marginal but stable, customer wants a tech, right answer is monitor and set expectations); *DHCP lease / CGNAT confusion* (not RF at all).
- Authoring guide in `packages/cases/README.md`.

Later (not planned in detail): cohorts and assignment for training leads; LLM-graded free text; case-authoring UI; case import/export.

14. Open decisions

1. New repo vs. inside existing monorepo. (Rec: new repo.)
2. Name vendors in copy or stay generic. (Rec: generic in UI, precise in gotchas.)
3. Free-text grading: keyword rubric v1 → LLM v2. (Rec: yes; design rubric as sentences now.)
4. Chart layer: hand-rolled SVG primitives vs. a library. (Rec: primitives; reconsider if `LinkCapacityChart` gets painful.)
5. Anonymous demo scope: stage 1 only, or stages 1–3? (Rec: stage 1; the commit-before-reveal hook is the demo.)
6. Whether reference screenshots come from the real employer systems or the vendor's public demo. (Either is fine for private reference; the latter avoids any question.)

15. Definition of "v1 done"

- A stranger can open `noisefloor.ca` , play stage 1 of case 001 without logging in, and understand what the site is.
 - A logged-in trainee can complete case 001 end-to-end, see a scored path with a revision bonus, read an annotated debrief, and see ten gotchas indexed.
 - No real customer, employer, or vendor-owned data anywhere in the repo or the deployed site.
 - Every dashboard renders from `world` ; changing one number in case 001 changes the chart.
 - CI runs unit tests, the synthetic-data check, and visual snapshots for every dashboard component.
-

Founding brief written 2026-09-16. Supersede with `openspec` proposals as they land; keep §3, §4 and §12 as standing principles unless deliberately changed.